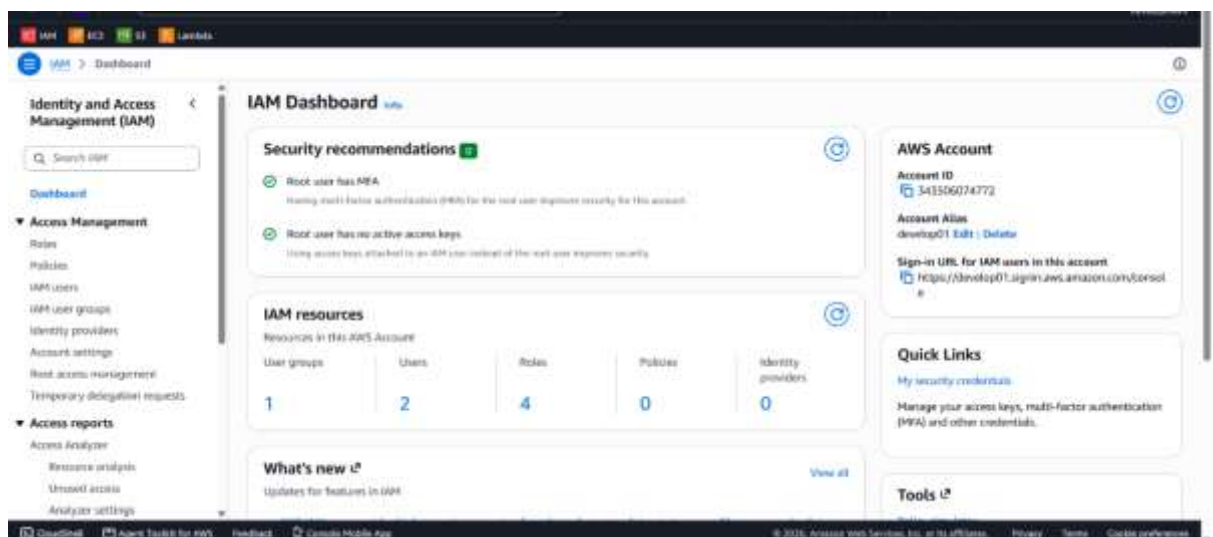
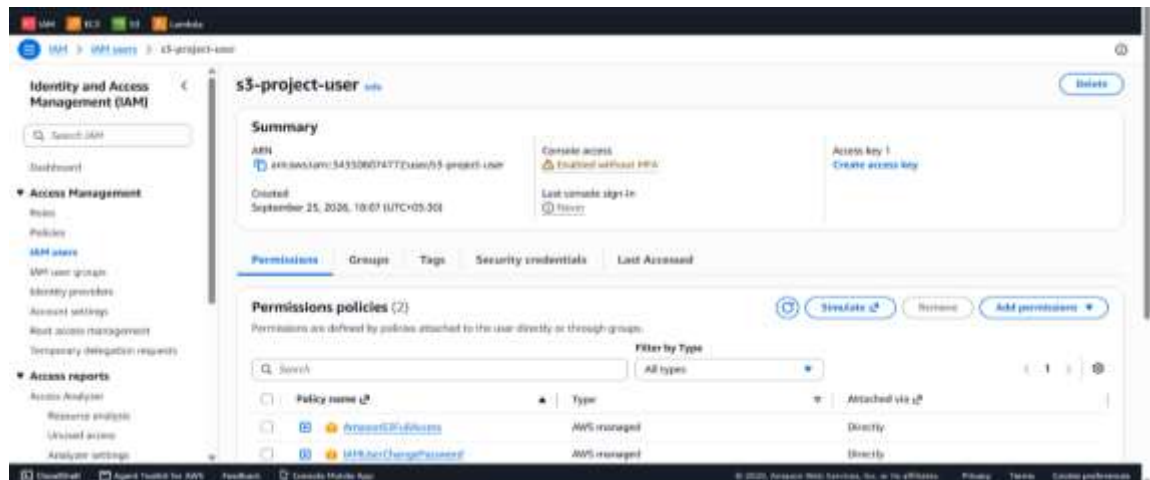


Mini Project: IAM & S3 Hands-On

1. Created IAM user with specific permissions

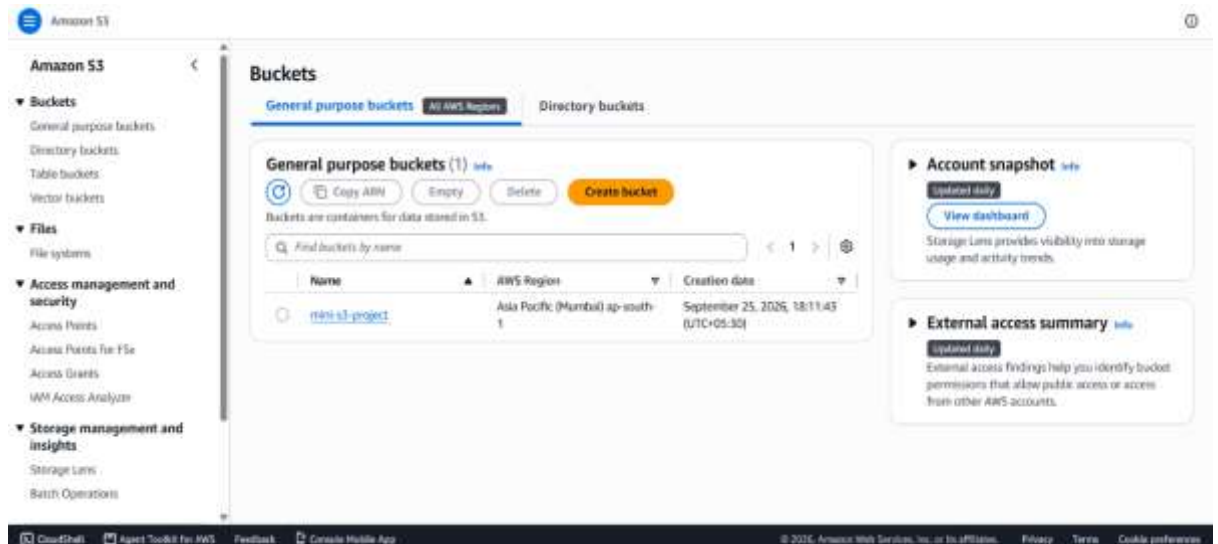
- **Steps:** AWS Management Console → IAM → IAM users → Create user
 - Name: s3-project-user
 - Custom password
 - Attached policy directly: AmazonS3FullAccess
 - Created
- **Why:** Attached only AmazonS3FullAccess instead of Administrator Access, to follow the principle of least privilege — giving the user only the permissions needed for managing S3, not full account access.
- **Screenshot:**

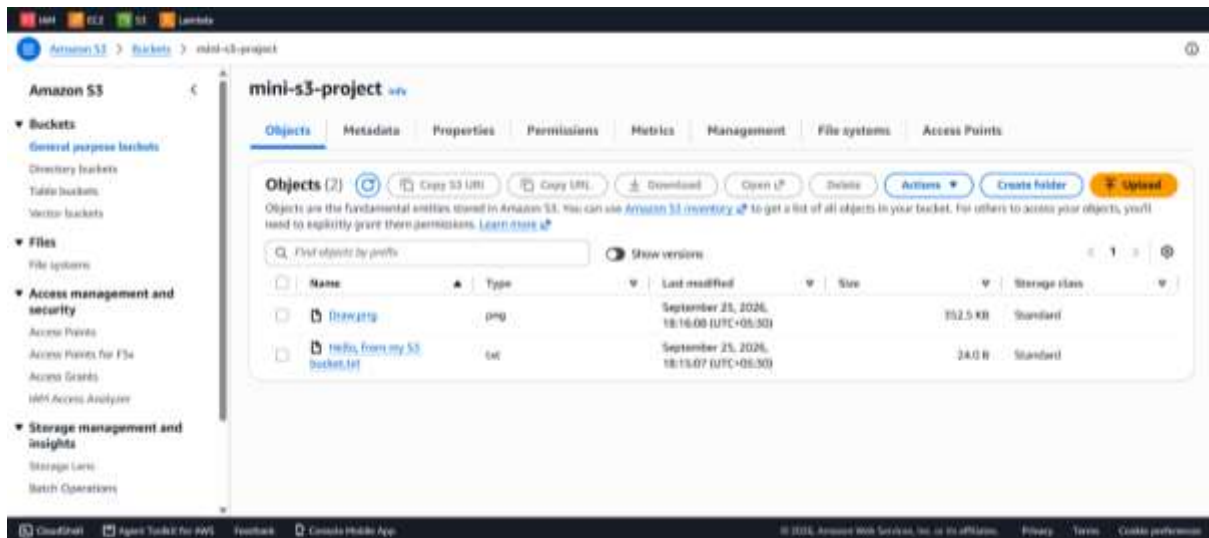




2. Created an S3 bucket and uploaded files

- **Steps:** Amazon S3 → Create bucket
 - Name: mini-s3-project
 - Region: Asia Pacific (Mumbai)
 - Unchecked "Block all public access" and acknowledged the warning
 - Created bucket
- **Then:** Opened the bucket → Upload → Add files → Upload
 - Uploaded 2 files: "Hello, from my S3 bucket.txt" and "Draw.png"
 - Used two different file types to test how S3 serves them publicly (text displays directly, image loads as an image)
- **Screenshot:**





3. Configured public access and tested the bucket policy

- **Steps:** Bucket → Permissions tab → Bucket policy → Edit
- Pasted a policy allowing public "s3:GetObject" access:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mini-s3-project/*"
    }
  ]
}
```

- Saved changes

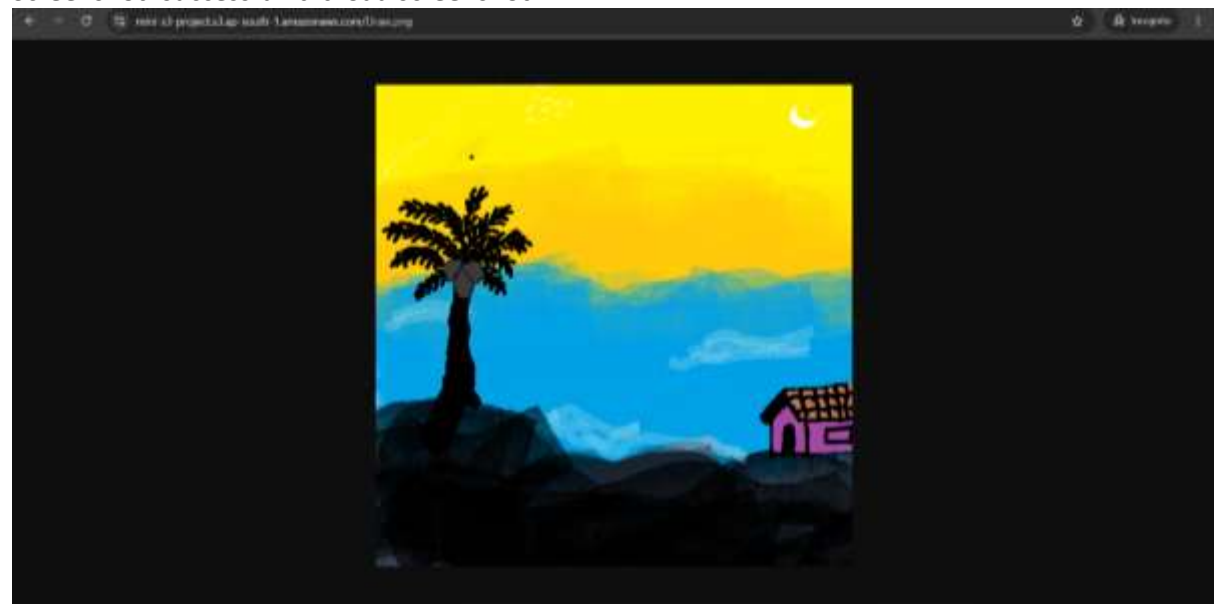
- **First test:** Opened the file's Object URL in a browser.
- **Result:** Access Denied error.
- **Why it failed:** The bucket policy alone was not enough. S3's "Block Public Access" setting at the bucket level overrides any bucket policy unless it is also turned off. This is a safety feature AWS built in on purpose, so buckets don't become public by accident.
- **Fix:** Went to Permissions → Block public access (bucket settings) → Edit
 - Unchecked "Block all public access"
 - Acknowledged the warning
 - Saved changes

- **Second test:** Reopened the same Object URLs.
- **Result:** Both files loaded successfully and publicly, with no AWS login required — the .txt file displayed as plain text, and Draw.png displayed as an image.

Screenshot: Access Denied screenshot



Screenshot: successful .txt load screenshot



Screenshot: successful Draw.png load screenshot



4. Locked the bucket back down (cleanup)

- **Steps:** Bucket → Permissions → Bucket policy → Edit - Deleted the bucket policy - Saved changes
- **Then:** Permissions → Block public access → Edit
 - Checked "Block all public access"
 - Saved and confirmed
- **Final test:** Reopened the same URLs
- **Result:** Access Denied returned again, confirming the bucket was secured

Learning Outcome

Through this project, I learned that IAM lets you scope a user's permissions to only what they need, rather than giving full account access. I also learned that S3 has two separate layers of access control — the bucket policy and the Block Public Access setting — and both must allow public access for a file to actually be reachable publicly. This showed me why AWS blocks public access by default, and the importance of making resources public only temporarily for testing, then locking them back down immediately afterward.